Experiment 3: Train and Save ML Model (Logistic Regression)

Aim

To train machine learning classification models using real-world datasets (Iris dataset for Logistic Regression) and save them for later use.

Software Requirements

- Python 3.10+
- scikit-learn
- pandas
- numpy
- pickle

Theory

Machine Learning models learn patterns from training data to make predictions on unseen data. Logistic Regression is a linear model for binary/multiclass classification. The Iris dataset contains measurements of iris flowers (150 samples, 4 features, 3 species). We serialize trained models using pickle to reuse them without retraining, which is essential for production deployment.

Procedure

Step 1: Create Project Directory

```
mkdir exp3_ml_training cd
exp3_ml_training
```

Step 2: Create and Activate Virtual Environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Step 3: Install Required Packages

```
pip install scikit-learn pandas numpy
```

Step 4: Create train_logistic_regression.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split from
sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix import
pickle
import pandas as pd

# Load the Iris dataset
print("Loading Iris dataset...") iris
= load_iris()
X = iris.data  # Features: sepal length, sepal width, petal length, petal width y =
iris.target  # Target: species (0=setosa, 1=versicolor, 2=virginica)

# Display dataset information
print(f"\nDataset shape: {X.shape}")
print(f"Number of classes: {len(iris.target_names)}")
print(f"Classes: {iris.target_names}") print(f"Feature
names: {iris.feature_names}")

# Create DataFrame for better visualization
df = pd.DataFrame(X,
columns=iris.feature_names) df['species'] = y
print("\nFirst 5 rows of dataset:")
print(df.head())

# Split data into training and testing sets (80-20 split) X_train,
X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\nTraining set size: {X_train.shape[0]}")
print(f"Testing set size: {X_test.shape[0]}")

# Train Logistic Regression model
print("\nTraining Logistic Regression model...")
model = LogisticRegression(max_iter=200, random_state=42)
model.fit(X_train, y_train)

# Make predictions on test set
y_pred = model.predict(X_test)
```

```
# Evaluate model
accuracy = accuracy_score(y_test, y_pred)
print(f"\nModel Accuracy: {accuracy * 100:.2f}%")

print("\nClassification  Report:")
print(classification_report(y_test,  y_pred,  target_names=iris.target_names))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Save the model
model_filename = 'iris_logistic_regression.pkl'

with open(model_filename, 'wb') as file: pickle.dump(model, file)

print(f"\nModel saved as '{model_filename}'")

# Save feature names for later use
with open('feature_names.pkl', 'wb') as file:
pickle.dump(iris.feature_names, file)

#  Save target names
with open('target_names.pkl', 'wb') as file:
pickle.dump(iris.target_names, file)

print("Feature names and target names saved.")

# Test loading the model print("\nTesting model
loading...")
with open(model_filename, 'rb') as file:
loaded_model = pickle.load(file)

# Make a sample prediction
sample = [[5.1, 3.5, 1.4, 0.2]]
# Sample measurements
prediction = loaded_model.predict(sample)
print(f"\nSample prediction for {sample}:")
print(f"Predicted class: {iris.target_names[prediction[0]]}")
```

Step 5: Run the Training Scripts

Train Logistic Regression:

```
python train_logistic_regression.py
```

Step 7: Verify Model Files

Check that these files were created:

```
ls -la *.pkl
```

You should see:

- `iris_logistic_regression.pkl`
- `feature_names.pkl`
- `target_names.pkl`

Step 8: Create test_models.py (Optional Testing Script)

```
import pickle


model_name = "Logistic Regression" model_file =
"iris_logistic_regression.pkl"


# Load supporting files
```

```
with open('feature_names.pkl', 'rb') as f:
        feature_names = pickle.load(f)
with open('target_names.pkl', 'rb') as f:
        target_names = pickle.load(f)

print("Feature Names:", feature_names)
print("Target Names:", target_names)

# Test samples test_samples = [
                [5.1,  3.5,  1.4,  0.2],   # Likely Setosa
                [6.7,  3.0,  5.2,  2.3],   # Likely Virginica
                [5.7,  2.8,  4.1,  1.3]    # Likely Versicolor
                ]
print(f"\n{'='*50}")
print(f"Testing {model_name}")
print('='*50)

with open(model_file, 'rb') as f:
      model = pickle.load(f)
for i, sample in enumerate(test_samples, 1):
      prediction = model.predict([sample])
       print(f"Sample {i}: {sample}")
       print(f"  Predicted: {target_names[prediction[0]]}")
```

Run the test:

```
python test_models.py
```

Output

- Two trained ML models saved as pickle files
- Training accuracy and evaluation metrics displayed
- Models can predict iris species from flower measurements
- Logistic Regression typically achieves ~95-97% accuracy

Screen-shots

Logistic Regression Execution:

```
personal/lab_manual/exp3_ml_training
venv ❯ python train_logistic_regression.py
Loading Iris dataset...

Dataset shape: (150, 4)
Number of classes: 3
Classes: ['setosa' 'versicolor' 'virginica']
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

First 5 rows of dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  species
0                5.1               3.5                1.4               0.2        0
1                4.9               3.0                1.4               0.2        0
2                4.7               3.2                1.3               0.2        0
3                4.6               3.1                1.5               0.2        0
4                5.0               3.6                1.4               0.2        0

Training set size: 120
Testing set size: 30

Training Logistic Regression model...

Model Accuracy: 96.67%

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30


Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]

Model saved as 'iris_logistic_regression.pkl'
Feature names and target names saved.

Testing model loading...

Sample prediction for [[5.1, 3.5, 1.4, 0.2]]:
Predicted class: setosa
```

Stored model `.pkL` files:

```
personal/lab_manual/exp3_ml_training
venv ❯ ls -la *.pkl
-rw-r--r-- 1 tejesh tejesh  94 Nov 16 22:32 feature_names.pkl
-rw-r--r-- 1 tejesh tejesh 836 Nov 16 22:32 iris_logistic_regression.pkl
-rw-r--r-- 1 tejesh tejesh 263 Nov 16 22:32 target_names.pkl
```

Testing Models:

```
personal/lab_manual/exp3_ml_training
venv ❯ python test_models.py
Feature Names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target Names: ['setosa' 'versicolor' 'virginica']

==================================================
Testing Logistic Regression
==================================================
Sample 1: [5.1, 3.5, 1.4, 0.2]
  Predicted: setosa
Sample 2: [6.7, 3.0, 5.2, 2.3]
  Predicted: virginica
Sample 3: [5.7, 2.8, 4.1, 1.3]
  Predicted: versicolor
```

Result

Successfully trained and saved two classification models (Logistic Regression and Naive Bayes) using the Iris dataset, achieving high accuracy. Models are serialized and ready for deployment in web applications.